Lab 3: Component Modelling & Architectural Pattern Selection

Written Justification: Self-Service Coffee Kiosk System

Architecture Selection

We chose **Layered Architecture** for the Self-Service Coffee Kiosk System.

Architectural choice: The kiosk is organised into three horizontal layers. The Presentation Layer holds the User Interface Component (touch screen). The Business Layer holds the Order Manager and Payment Service Components. The Data & Hardware Access Layer holds the Receipt Printer and Menu Database Components. Components communicate through the interfaces shown in the component diagram, including the Order Interface, Payment Processing API, Receipt Printing Interface, and Database Query Interface.

Two Reasons for This Choice

1. The kiosk is one small, self-contained system. It offers only 3 coffees, 2 sizes and card-only payment, and runs on a single machine in the café. Microservices would add service discovery, network calls and distributed data consistency that a system this size does not need, and a client-server design would add a central server that the kiosk does not require.

2. The layers map directly onto the requirements. Touch-screen handling sits in the Presentation Layer. Order flow, pricing and payment logic sit in the Business Layer. Printer hardware and menu/pricing storage sit in the lowest layer. Each component can be replaced behind its interface (for example a different printer model or payment provider) without changing the screens, and each layer can be tested on its own.

Security Advantage

The User Interface never talks to the Payment Service, the printer or the database directly. Every request passes through the Order Manager, which gives one controlled entry point where inputs are validated (only valid coffee types and sizes are accepted). Credit-card details are handled only inside the Payment Service and are never stored in the Menu Database or the UI, which reduces the attack surface. Menu and pricing data cannot be changed from the touch screen.

Performance Benefit

All layers run on the same kiosk, so calls between components are local interface calls with no network round trip. The touch screen therefore responds quickly. The menu and prices are small, so the Order Manager can read them once through the Database Query Interface and reuse them while the customer is ordering, instead of querying the database on every tap.

Alternatives Considered

Microservices: rejected because of operational complexity and network latency for such a small scope.

Client-Server: rejected because a central server is a single point of failure and a scalability bottleneck, while the kiosk must keep taking orders on its own.